

EXPERIMENT 5

Student Name: Hunar Katyal

UID: 24BDA70085

Branch: AIT-CSE

Section/Group: 24AIT_KRG-G2

Semester: 5

Subject Name: ADBMS

Subject Code: 24CSP-333

Experiment 5 -- To study and implement advanced SQL queries using Joins, Correlated Query's and Conditional statement.

Aim : To study and implement correlated subqueries in SQL by retrieving employees whose salaries are among the top three highest distinct salary values within their respective departments, while correctly handling duplicate salary values using aggregate functions and joins

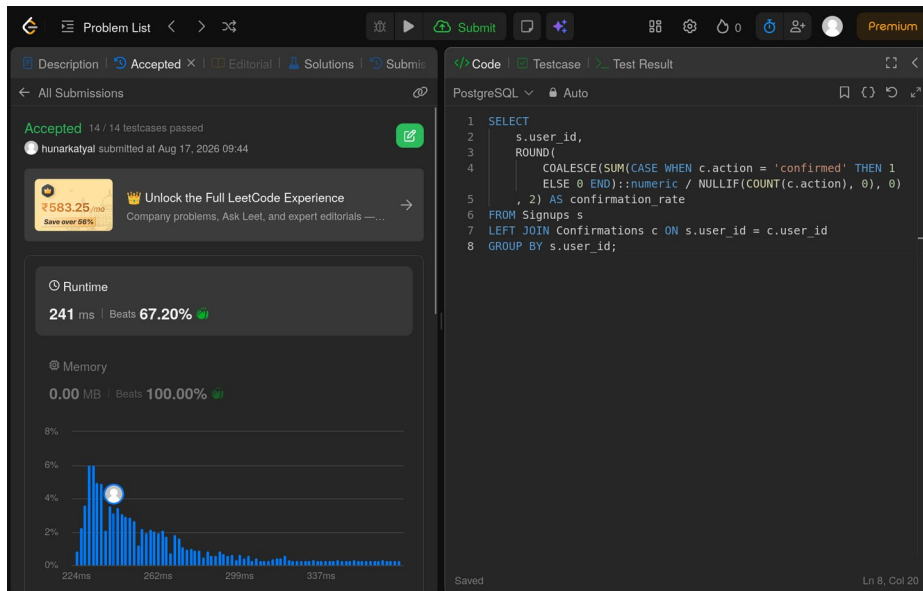
Question :5 Part A (i): Calculate Confirmation Rate

SQL Query:

```
SELECT p.user_id, round(avg(CASE WHEN s.action like 'confirmed' THEN 1 ELSE 0 END),2) as  
confirmation_rate  
FROM confirmations as s  
right JOIN signups as p  
ON s.user_id = p.user_id  
group by p.user_id
```

EXPERIMENT 5

4



Question: Part A (ii) Calculate Cancellation rate

SQL Query:

```
SELECT tp.request_at as "Day", ROUND(AVG(CASE WHEN tp.status like 'cancelled%' THEN 1 ELSE 0
END),2)as "Cancellation Rate"
```

```
FROM trips as tp
```

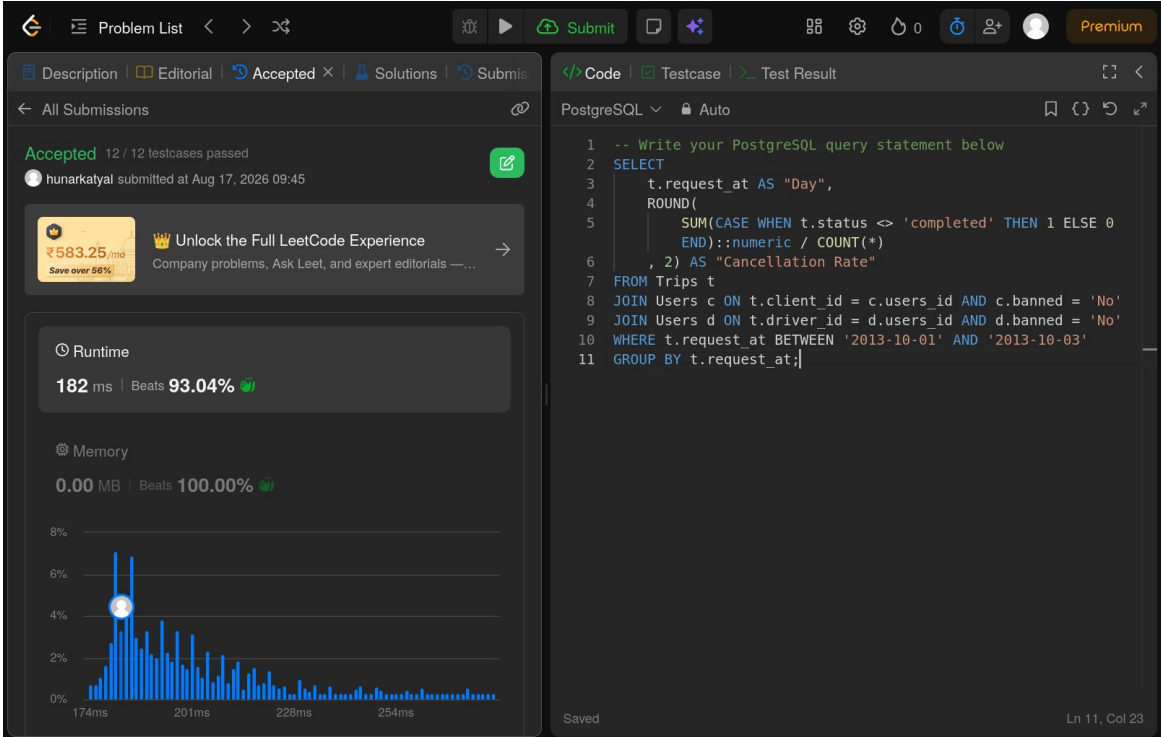
```
JOIN Users as c ON c.users_id = tp.client_id
```

```
JOIN Users as d ON d.users_id = tp.driver_id
```

```
WHERE c.banned = 'No' AND d.banned = 'No' AND tp.request_at BETWEEN '2013-10-01' AND '2013-10-03'
```

```
GROUP BY tp.request_at
```

EXPERIMENT 5



The screenshot shows a LeetCode submission for a PostgreSQL query problem. The submission is accepted, showing runtime of 182 ms and memory of 0.00 MB. The query is a complex SQL statement using joins and aggregate functions.

```
1 -- Write your PostgreSQL query statement below
2 SELECT
3     t.request_at AS "Day",
4     ROUND(
5         SUM(CASE WHEN t.status <> 'completed' THEN 1 ELSE 0
6             END)::numeric / COUNT(*)
7     , 2) AS "Cancellation Rate"
8 FROM Trips t
9 JOIN Users c ON t.client_id = c.users_id AND c.banned = 'No'
10 JOIN Users d ON t.driver_id = d.users_id AND d.banned = 'No'
11 WHERE t.request_at BETWEEN '2013-10-01' AND '2013-10-03'
12 GROUP BY t.request_at;
```

Learning Outcome

After completing this experiment, the learner will be able to:

1. **Understand and implement correlated subqueries** to solve SQL problems involving row-wise comparisons.
2. **Apply aggregate functions** such as COUNT() and DISTINCT to retrieve top-N records within groups.
3. **Use joins and filtering techniques** to combine data from multiple tables and generate meaningful reports.
4. **Analyze and solve complex SQL queries** involving grouped data, ranking logic, and handling duplicate values effectively.